Group Communication

6.1 Unicasting versus Multicasting

In the IPC data is sent from a source process, thesender, to one destination process, the receiver. This form of IPC can be calledunicasting, the sending of information to a single receiver, as opposed to multicasting, the sending of information to multiple receivers. Unicasting provides one-to-one IPC; multicasting supports one-to-many IPC.

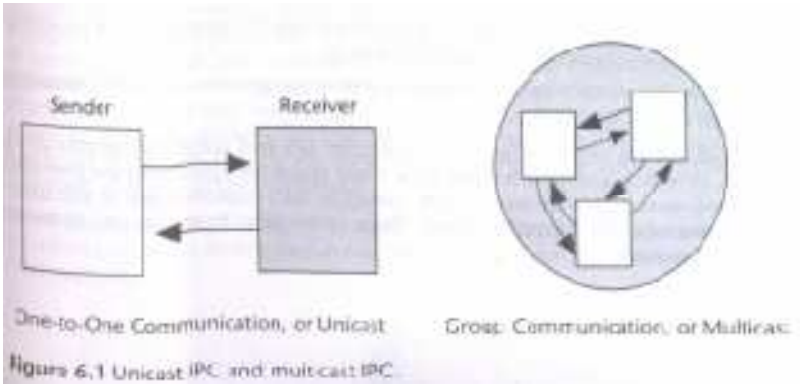


Figure 6.1 Unicast IPC and multicast IPC

Whereas the majority of network services and network applications use unicasting for IPC, multicasting is useful for applications such as instant messages,groupware, online conferences, and interactive learning, and it can be used forapplications such as a real-time online auction. Multicasting is also useful in thereplication of services for fault tolerance.

In an application or network service that makes use of multicasting, a set ofprocesses forms a group, called a multicast group. Each process in a group cansend and receive messages. A message sent by any process in the group can bereceived by each participating process in the group. A process may also chooseto leave a multicast group.

In an application such as online conferencing, a group of processes interoperateusing multicasting to exchange audio, video, and/or text data.

6.2 An Archetypal Multicast API

An application program interface that supports multicasting must provide thefollowing primitive operations:

• Join. The join operation allows a process to join a specific multicast group.A process that has joined a multicast group is a member of the group and isentitled to receive all messages addressed to the group. A process may be amember of one or more multicast groups at anyone time. Note that for thisand other multicast operations, a naming scheme is needed to uniquely identifya multicast group.

• Leave. This operation allows a process to stop participating in a multicastgroup. A process that has left a multicast group is no longer a member of thegroup and is thereafter not entitled

to receive any multicast addressed to thegroup, although the process may remain a member of other multicast groups.

• Send. The send operation allows a process to send a message to all processescurrently participating in a multicast group.

• Receive. The receive operation allows a member process to receive messagessent to a multicast group.

6.3 Connection less versusConnection-Oriented Multicast

A basic multicast mechanism is connectionless. The reason is obvious if youconsider the one-to-many nature of multicasting. In a group of *n* processes, if aconnection is to be established between a sender and every other process in thegroup, a total of *n-1*connections will be needed. Moreover, each of the *n*processesmay potentially be a sender, so that each process must maintain a connectionwith every other process, resulting in a total of $n * (n - 1)$ or roughly n^2connections. If *n-1* is large, the sheer number of such connections will becomeprohibitively expensive.

Moreover, connection less IPC is appropriate for a common type of multicastapplications: applications where audio or video data is transmitted amongprocesses in real time. (By saying that the data is transmitted in real time, wemean that the latency between sending and receiving should be close to zero.)

When audio or video data is transmitted, the reduction in latency provided byconnectionless communication outweighs the advantages offered by connection-oriented communication. When data for animation is sent, for example, itis more acceptable for a receiver to experience a distortion in the image of anoccasional frame (a likely occurrence with connectionless communication) thana frequent, perceptible delay between consecutive frames (a likely occurrencewith connection-oriented communication).

6.4 Reliable Multicasting versusUnreliable Multicasting

When a multicast message is sent by a process, the run-time support of the multicastmechanism is responsible for delivering the message to each process currently in the multicast group. As each participating process may reside on aseparate host, the delivery of these messages requires the cooperation of mechanismsrunning independently on those systems. Due to factors such as failuresof network links, defects in network hosts, routing delays, and differences in softwareand hardware, the time between when a unicast message is sent and whenit is received may vary among the recipient processes. While the differencesin message delivery to individual hosts may be insignificant if the machines are localized geographically, this may not be the case if the hosts are dispersed over a wide-area network.

Moreover, a message may never be received by one or more of the processes at all owing to errors and/or failures in the network the machines, or the runtime support. Whereas some applications, such as video conferencing, can tolerate an occasional miss or misordering of

messages, there are applications-such as database applications for which such anomalies are unacceptable.

Therefore, when employing a multicasting mechanism for an application, it isimportant that you choose one with the characteristics appropriate for yourapplication. Otherwise, measures will need to be provided in the logic of theapplication in order to handle the anomalies that may occur in message delivery.

For that reason, it is useful to classify multicasting mechanisms in terms of theircharacteristics of message delivery.

Unreliable Multicasting

At its most basic, a multicast system will make a good-faith attempt to delivermessages to each participating process, but the delivery of the correct messageto each process is not guaranteed. Thus any message sent by a process may bereceived by zero or more processes. In the best case, the message, in its correctform, is received by all processes. In the worst case, the message may be receivedby no process correctly. In other cases, the message may be received by some butnot all processes, or the message may be received by some processes in a corruptedform, or more than once. Such a system is said to provide unreliablemulticasting.

Reliable Multicasting

A multicast system that guarantees that each message is eventually deliveredcorrectly to each process in the group is said to provide reliable multicasting.

In such a system, each message sent by a process can be assumed to be deliveredin a noncorrupted form to all processes in the group eventually.

The definition of reliable multicasting requires that each participating processreceives *exactly one copy* of each message sent. Yet the definition places *no* restrictionon the order in which the messages are delivered to each individualprocess: Each process may receive the messages in any permutation of the orderin which those messages were sent. For applications where the order of message delivery is significant, it is helpful to further classify reliable multicast systemsas described below.

Unordered

An unordered reliable multicast system guarantees the safe delivery of each message,but it provides *no guarantee* of the delivery order of the messages.

For example, suppose three processes P1, P2and P3 have formed a multicast group. Further suppose that three messages, m1,m2 and m3 been sent to thegroup. Then an unordered reliable multicast system may deliver the messages to each of the three processes in any of the 3! (factorial of 3)= 6 permutations.

Note that it is possible for each participant to receive the messages in an orderdifferent from the order of messages delivered to other participants. In ourexample, it is possible for the messages to be delivered to P1 in the order ml-m2-m3 for the messages to be delivered to P2 in the order m2-ml-m3, and for themessages to be delivered to P3in the order m1-m3-m2z. Of course, it is also possiblefor the messages to be delivered to P_l,P2, and P_3 in the same order, say m1-m2-m3but we cannot make that assumption about an application if it employsan unordered multicast mechanism.

FIFO Multicasting

A system that guarantees that the delivery of the messages adheres to thefollowing condition is said to provide FIFO (first in-first out) or send-ordermulticast:

If process P sent messages m_i*and* m_j, *in that order, to a multicast group, theneach process in the multicast group will be delivered the messages* m_i*and* m_j*inthat order.*

Toillustrate this definition, let's look at an example. Suppose P1 sends messages m1,m2,m3 in that order, to a multicast group G. Then, with FIFO multicasting,each process in G is guaranteed to have those messages delivered in thatsame order: m1,m2 and then m3.Note that this definition places no restriction onthe delivery order among messages sent by different processes.

Causal-Order Multicasting

A multicast system is said to provide causal multicasting if its message deliverysatisfies the following criterion:

> *If message* m_i*causes (results in) the occurrence of message* m_j, *then* m_i*will bedelivered to each process prior to* m_j.*Messages* m_i*and* m_j*are said to have acausal or happen-before relationship, denoted* $m_i \rightarrow m_j$.

The happen-before relationship is transitive: if $m_i \rightarrow m_j$and $m_j \rightarrow m_k$then$m_i \rightarrow m_j \rightarrow m_k$. In this case, a causal-order multicast system guarantees that thesethree messages will be delivered to each process in the order of m_i, m_j, *then* m_k;

Atomic Order Multicasting

In an atomic-order multicast system, all messages are guaranteed to be deliveredto each participant in the *exact same order.* Note that the delivery order doesnot have to be FIFO or causal, but it must be the same for each process.

Example:

P1 sends m1, P2 sends m2, and P3 sends m3.

An atomic system will guarantee that the messages will be delivered to eachprocess in only one of the six orders: m1-m2-m3, m1-m3-m2, m2-m1-m3, m2-m3-m1, m3-m1-m2, m3-m2-m1

6.5 The Java Basic Multicast API

At the transport layer, the basic multicast supported by Java is an extension ofthe User Datagram Protocol (UDP), which, as you recall, is connectionless andunreliable. At the network layer of the network architecture, multicast packetsare transmitted across networks using Internet multicast routing supported by routers (known as mrouters) capable of multicast routing in additionto unicast routing. Multicasting on a local network (one interconnectedwithout a router) is carried out using multicasting supported by the local areanetwork protocol (such as Ethernet multicasting). The routing and delivery ofmulticast messages are topics beyond the scope of this book. Fortunately, suchmatters are transparent to an application programmer using a multicast API.

For the basic multicast API, Java provides a set of classes that are closely relatedto the datagram socket API classes. There arethree major classes in the API, the first three of which we have already seen inthe context of datagram sockets.

1. ***InetAddress.***In the datagram socket API, this class represents the IP address ofthe sender or receiver. In multicasting, this class can be used to identify amulticast

2. ***DatagramPacket.***As with datagram sockets, an object of this class represents an actual datagram; in multicasting, a *DatagramPacket* object represents apacket of data sent to all participants or received by each participant in amulticast group.

3. ***MulticastSacket.***The *MulticastSocket* class is derived from the *DatagramSocket*class, with additional capabilities for joining and leaving a multicast group.

An object of the multicast datagram socket class can be used for sending andreceiving IP multicast packets.

IP Multicast Addresses

Wecall that in the Java unicast socket API, a sender identifies a receiver by specifying the host name of the receiving process as well as the protocol port to which the receiving process is bound.

Consider for a moment what a multicast sender needs to address. Instead of aSingle process, a multicast datagram is meant to be received by all the processesthat are currently members of a specific multicast group. Hence each multicastdatagram needs to be addressed to a multicast group instead of an individualprocess.

The Java multicast API uses the Internet Protocol (IP) multicast addresses foridentifying multicast groups.

In IPv4, a multicast group is specified by (1) a class D IP address *combined with*(2) a standard UDP port number.

IPv4 multicast addresses are managed and assigned by the InternetAssigned Numbers Authority (IANA).

An application that uses the Java multicast API must specify at least one multicastaddress for the application. To select a multicast address for an application,there are the following options:

1. Obtain a permanently .assigned static multicast address from IANA.Permanent addresses are limited to global, well-known Internet applications,and their allocations are highly restricted.

2. Choose an arbitrary address, assuming that the combination of the chosenrandom address and port number will not likely be in use.

3. Obtain a transient multicast address at run time; such an address can be received by an application through the Session Announcement Protocol.

Joininga Multicast Group

To join a multicast group at IP address *m* and UDP port *p,* a *MulticastSocket* objectmust be instantiated with *p,* then the object's *joinGroup* method can be invoked,specifying the address *m:*

*//*join a Multicast group at IP address 239.1.2.3 and port 3456

```
InetAddress group = InetAddress.getByName("239.1.2.3");

MulticastSocket s = new MulticastSocket(3456);

s.joinGroup(group);
```

Sending to a Multicast Group

A multicast messagecan be sent using syntax similar to that for the datagramsocket API. Specifically, a datagram packet must be created with the specificationof a reference to a byte array containing the data, the length of the array,the multicast address, and a port number. The *send* method of the*MulticastSocket* object (inherited from the *DatagramSocket* class) can then beinvoked to send the data.

It is not necessary for a process to join a multicast group in order to send messagesto it; although it must do so in order to receive the messages. When a messageis sent to a multicast group, all processes that have joined the multicastgroup, which may include a sender, can be expected (but not guaranteed) toreceive the message.

Receiving Messages Sent to a Multicast Group

A process that has joined a multicast group may receive messages sent to thegroup using syntax similar to that for receiving data through a datagram socketAPI.

Leaving a Multicast Group

A process may leave a multicast group by invoking the *leaveGroup* method of a*MulticastSocket* object, specifying the multicast address of the group:

```
s.leaveGroup(group);
```

Setting the "Time-to-Live"

The run-time support for a multicast API often employs a technique known asmessage propagation, whereby a packet is propagated from a host to a neighboringhost in an algorithm that, when executed properly, will eventuallydeliver the message to all the participants. Under some anomalous circumstances,however, it is possible that the algorithm that controls the messagepropagation does not terminate properly, resulting in a packet circulating in thenetwork indefinitely. This phenomenon is undesirable, as it causes unnecessaryoverhead on the systems and the network. To avoid this occurrence, it is recommendedthat a "time-to-live" parameter be set with each multicast datagram.

The time-to-live (ttl) parameter, when set, limits the count of network links, orhops, to which the packet will be forwarded on the network.

6.6 Reliable Multicast API

TheJava multicast API that we have explored in this chapter is an extension ofthe datagram socket API. As a result, it shares a key characteristic of datagrams:unreliable delivery.

In particular, messages are not guaranteed to be delivered to any of the receiving processes. Hence, the API provides unreliable multicasting. But keep in mind that in your exercises, if you run your processes on onehost or on .hosts on one subnet, you are not likely to observe any loss of messagesorscrambling in the delivery order of the messages. These anomalies aremore likely when the participating hosts are remotely connected, due to networkfailures or routing delays.

Asmentioned, there are applications for which unreliable multicasting is unacceptable. For such applications, there are a number of available packages that providereliable multicast API, some of which include

- The ***Java Reliable Multicast Service (JRM Service)***is a package that enhances the Java basic multicastPI by providing the capabilities for a receiver to repair multicast data thatare lost or damaged, as well as security measures to protect data privacy.
- ***The Totem system***, developed by the University of California, Santa Barbara, provides reliable, totally ordered delivery of messagesto processes within process groups on a single local-area network, or overmultiple local-area networks interconnected by gateways.
- TASC's ***Reliable Multicast Framework(RMF)*** provides reliable and send-ordered (FIFO) multicasting.